

Question 1 — Cookies vs Sessions in PHP

Cookies and Sessions are both mechanisms used in PHP to **persist data across multiple HTTP requests**, because HTTP is a stateless protocol. However, they work very differently. The fundamental difference is **where the data is stored**: cookies are stored on the **client (browser)**, while session data is stored on the **server**.

The table below compares them across the requested criteria, followed by a short explanation of each point.

Criteria	Cookies	Sessions
Storage location	Stored on the client side , inside the user's browser. The data travels with every HTTP request in the Cookie header.	Stored on the server side (e.g. a file in PHP's <code>session.save_path</code>). Only a small session ID is kept on the client as a cookie.
Lifetime	Long-lived / persistent . Expiry is set manually with <code>setcookie()</code> . With no expiry, the cookie lasts only until the browser closes. Can survive days, months or years.	Short-lived / temporary by default. Lasts until the browser closes or the session times out (<code>session.gc_maxlifetime</code>). Destroyed with <code>session_destroy()</code> .
Security	Less secure . Data sits on the user's machine and is visible/editable by the user. It is sent over the network on every request and can be stolen (XSS) or tampered with. Do not store sensitive data here.	More secure . The actual data never leaves the server; only the session ID is exposed. Harder to read or modify. Still protect against hijacking/fixation (HTTPS, <code>httponly</code> , regenerate IDs).
Data size limitations	Small . About 4 KB per cookie , and browsers limit the number of cookies per domain (~20–50). Not suitable for large data.	Large . Limited mainly by server memory/disk , so it can store much more data (objects, arrays, large datasets).
Typical usage scenarios	"Remember me" login, user preferences (language, theme), tracking/analytics, storing non-sensitive long-term data.	Logged-in user state, shopping carts, temporary form data, CSRF tokens, and any sensitive or temporary per-user data during a visit.

Short explanations

- **Storage location** — A cookie's value physically lives in the browser and is attached to every request. A session's value lives on the server; the browser only holds a session identifier (PHPSESSID) used to look up the data server-side.

- **Lifetime** — Cookies can persist far beyond a single visit by setting an explicit expiry date, which is why they suit long-term preferences. Sessions are temporary and are cleared when the user closes the browser or after an inactivity timeout.
- **Security** — Because cookie data is on the client, the user can read or modify it and it is exposed on the network, making it the less secure choice. Sessions keep the real data on the server, so they are safer for sensitive information such as authentication state.
- **Data size limitations** — Cookies are capped at roughly 4 KB each, so only small pieces of data fit. Sessions are limited by server resources and can hold much larger, more complex data.
- **Typical usage scenarios** — Use **cookies** for small, non-sensitive, long-lived data like preferences or "remember me" tokens. Use **sessions** for sensitive or temporary per-user state like login status and shopping carts.

Simple PHP examples

```
// COOKIE – stored in the browser for 7 days
setcookie("theme", "dark", time() + (7 * 24 * 60 * 60));
echo $_COOKIE["theme"];    // read it back on later requests

// SESSION – stored on the server
session_start();
$_SESSION["username"] = "Ayse";    // saved server-side
echo $_SESSION["username"];        // available on every page until destroyed
```